



JAVASCRIPT

Basic to Advanced Notes

Complete handwritten notes for
beginners to advanced learners.

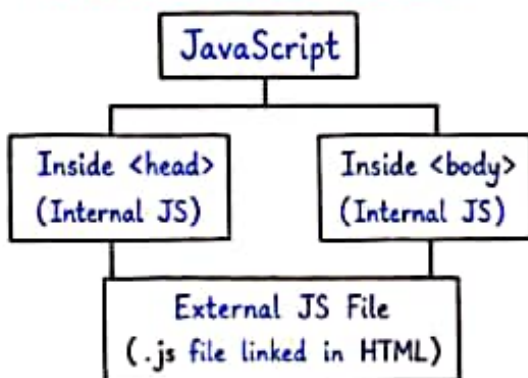


What is JavaScript?

JavaScript (JS) is a lightweight, interpreted programming language used to make web pages interactive and dynamic.



JavaScript in Web Pages



Example (Simple JavaScript Code)

```
// This is a simple JavaScript program
console.log("Hello World!");
let name = "JavaScript";
alert("Welcome to " + name + "!");

// Output:
// Hello World!
// Alert: Welcome to JavaScript!
```

Key Features

- * Easy to learn
- * Makes web pages interactive
- * No compilation needed
- * Works with HTML & CSS
- * Large community support
- * Cross-platform (Windows, Mac, Linux)
- * Supported by all modern browsers



★ Topics Covered

- Introduction to JavaScript
- Variables & Data Types
- Operators
- Control Statements
- Loops
- Functions
- Arrays
- Objects
- DOM (Document Object Model)
- Events
- BOM (Browser Object Model)
- ES6+ Features
- Async Programming
- Error Handling



Code.
Learn.
Build.
Repeat.



Page 1





JAVASCRIPT VARIABLES

Variables are used to store data values.
They act as containers for data.

KEY POINTS

- Variables store data values.
- Declare a variable before using it.
- Use meaningful variable names.
- JS is case sensitive.

1. Ways to Declare Variables

var

Old way to declare variables.
Function scoped.
Can be re-declared and updated.

let

Modern way to declare variables.
Block scoped.
Can be updated but not re-declared in the same scope.

const

Used to declare constants.
Block scoped.
Cannot be updated or re-declared.

Examples

```
// Using var
var name = "Shainaz";
console.log(name); // Shainaz

// Using let
let age = 21;
age = 22; // Allowed
console.log(age); // 22

// Using const
const city = "Navi Mumbai";
console.log(city); // Navi Mumbai
// city = "Mumbai"; // Error (Cannot reassign)
```

2. Variable Naming Rules

- ★ Can contain letters, digits, underscores (_) and \$.
- ★ Must start with a letter, _ or \$.
- ★ Cannot start with a number.
- ★ Case sensitive (myVar and myvar are different).
- ★ Cannot use reserved keywords.

Valid Examples

- name
- _age
- \$price
- user1
- firstName

Invalid Examples

- 1name (starts with number)
- user-name (hyphen not allowed)
- name@123 (special char not allowed)
- var (reserved keyword)

3. Best Practices

- ♥ Use meaningful and descriptive names.
- ♥ Use camelCase (firstName, totalPrice).
- ♥ Avoid very short names (x, y, z).
- ♥ Use const by default, let when value changes.
- ♥ Follow consistent naming style.

Quick Notes

- Use let and const in modern JavaScript.
- Avoid using var.
- Always declare variables before use.
- Good variable names make code readable and maintainable.

Mini Code Example

```
const firstName = "Shainaz";
let score = 85;
var isStudent = true;

score = 90; // Updated
isStudent = false; // Updated
console.log(firstName, score, isStudent);
// Output: Shainaz 90 false
```

Practice makes perfect!

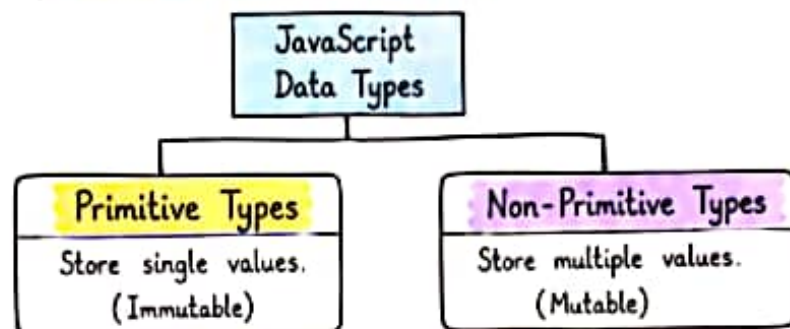


JAVASCRIPT DATA TYPES

Data types define the type of value a variable can hold in JavaScript.



1. Types of Data Types



KEY POINTS

- JavaScript has 7 primitive and 5 non-primitive data types.
- Primitive types store single values.
- Non-primitive types store multiple values.
- Use typeof to check data type.
- JS is dynamically typed (no need to declare type).



Primitive Data Types

- String → "Hello"
- Number → 25, 3.14
- Boolean → true, false
- Undefined → undefined
- Null → null
- Symbol → Symbol('id')
- BigInt → 123456789012345n

Non-Primitive Data Types

- Object → { name: "Shainaz" }
- Array → [1, 2, 3]
- Function → function() {}
- Date → new Date()
- RegExp → /abc/

Examples



```
console.log(typeof "Hello"); // string
console.log(typeof 25); // number
console.log(typeof true); // boolean
console.log(typeof undefined); // undefined
console.log(typeof null); // object (JS bug)
console.log(typeof [1,2,3]); // object
console.log(typeof {name:"Shainaz"}); // object
```

2. typeof Operator

Used to find the data type of a variable.

Syntax:

typeof variableName

3. null vs undefined vs NaN

Value	Meaning	typeof	Example
undefined	Variable declared but no value	undefined	let x;
null	Intentional empty value	object (JS bug)	let x = null;
NaN	Not a Number (result of invalid math)	number	let x = 0/0;



Quick Notes

- typeof null returns "object" (old JS bug).
- Arrays are objects in JavaScript.
- Use Array.isArray() to check for arrays.
- Functions are also objects (typeof returns "function").

Example Outputs

```
typeof "Hello" → string
typeof 25 → number
typeof true → boolean
typeof undefined → undefined
typeof null → object
typeof [1,2,3] → object
typeof function() {} → function
```





JAVASCRIPT OPERATORS

Operators are special symbols used to perform operations on values and variables.

KEY POINTS

- Operators work on one or more operands.
- They return a value.
- JS has many types of operators.

1. Types of Operators

Type	Description	Operators	Example
① Arithmetic	Used to perform mathematical operations.	+ - * / % ** ++ --	let a = 10, b = 3; console.log(a + b); // 13
② Assignment	Used to assign values to variables.	= += -= *= /= %= **=	let x = 5; x += 2; // x = 7
③ Comparison	Used to compare two values.	== === != !== > < >= <=	console.log(5 > 2); // true
④ Logical	Used to combine conditions.	&& !	console.log(true false); // true
⑤ Ternary	Short hand if-else operator.	? :	let age = 18; let res = age >= 18 ? "Adult" : "Minor";
⑥ Bitwise	Performs operations on binary numbers.	& ^ ~ << >> >>>	console.log(5 & 1); // 1
⑦ Type	Checks the type of a value.	typeof	console.log(typeof "Hello"); // string
⑧ Other	Other special operators.	in instanceof delete new void	console.log("name" in obj); // true

Example

```

let a = 10, b = 3;
// Arithmetic
console.log(a + b); // 13
console.log(a * b); // 30
// Comparison
console.log(a > b); // true
// Logical
console.log(a > 5 && b < 5); // true
// Ternary
let result = (a > b) ? "Yes" : "No";
console.log(result); // Yes

```

ARITHMETIC OPERATORS

- + : Addition
- : Subtraction
- * : Multiplication
- / : Division
- % : Modulus (Remainder)
- ** : Exponentiation (Power)
- ++ : Increment
- : Decrement

2. Increment & Decrement

Prefix

Value is changed first, then used.

```

let x = 5;
console.log(++x); // 6
console.log(x); // 6

```

Postfix

Value is used first, then changed.

```

let y = 5;
console.log(y++); // 5
console.log(y); // 6

```

Quick Notes

- Use === for strict equality (checks value + type).
- Use == for loose equality (only value).
- Ternary operator is useful for short conditions.
- Logical operators return boolean values.

3. Ternary Operator

A shorthand way to write if-else.

condition ? expression_if_true : expression_if_false

Example

```

let marks = 75;
let grade = (marks >= 60) ? "Pass" : "Fail";
console.log(grade); // Pass

```

4. Operator Precedence (High to Low)

Precedence decides which operator will be executed first.

() → ** → * / % → + - → < <= > >=

→ == != === !== → && → || → ? : → =

Use parentheses () to control precedence.



JAVASCRIPT CONTROL STATEMENTS

Control statements are used to control the flow of execution in a program based on conditions. ★

KEY POINTS

- They help in decision making.
- Code runs conditionally.
- Improve logic and program flow.
- Make programs dynamic and smart. ♥

1. if Statement

Executes a block of code if the condition is true.

```
if (condition) {  
  // code to be executed  
}
```

Example

```
let age = 18;  
if (age >= 18) {  
  console.log("You are an adult");  
}
```

Output

You are an adult

2. if...else Statement

Executes one block if condition is true, otherwise executes another block.

```
if (condition) {  
  // code if true  
} else {  
  // code if false  
}
```

Example

```
let num = 7;  
if (num % 2 == 0) {  
  console.log("Even Number");  
} else {  
  console.log("Odd Number");  
}
```

Output

Odd Number

3. if...else if...else Ladder

Used when there are multiple conditions. Checks conditions one by one.

```
if (condition1) {  
  // code  
} else if (condition2) {  
  // code  
} else {  
  // code  
}
```

Example

```
let marks = 75;  
if (marks >= 90) {  
  console.log("Grade: A");  
} else if (marks >= 60) {  
  console.log("Grade: B");  
} else {  
  console.log("Grade: C");  
}
```

Example Outputs

- Grade: A (marks 90 or more)
- Grade: B (marks 60 to 89)
- Grade: C (marks below 60)



4. switch Statement

Used when we have multiple options to choose from. It is a cleaner way instead of many if...else if.

```
switch(expression) {  
  case value1:  
    // code  
    break;  
  case value2:  
    // code  
    break;  
  default:  
    // code  
}
```

Example

```
let day = 3;  
switch(day) {  
  case 1: console.log("Monday");  
    break;  
  case 2: console.log("Tuesday");  
    break;  
  case 3: console.log("Wednesday");  
    break;  
  default: console.log("Invalid Day");  
}
```

Output

Wednesday

switch Note

- Use break; to stop execution after a case.
- If break is missing, next cases also run.
- default runs if no case matches. ♥

★ Quick Notes

- if is used for single condition.
- if...else for two conditions.
- if...else if...else for multiple conditions.
- switch is best when comparing one variable with multiple values. 😊



Pro Tip

Use control statements wisely to make your code efficient and easy to maintain. ♥



JAVASCRIPT LOOPS

Loops are used to execute a block of code multiple times.

KEY POINTS

- Loops save time and effort.
- They reduce repetitive code.
- JS has for, while, do...while
- Use break to stop a loop.
- Use continue to skip an iteration.

1. for Loop

Used when we know the number of times to repeat.

```
for ( initialization; condition; update ) {  
  // code to be executed  
}
```

Example

```
for (let i = 1; i <= 5; i++) {  
  console.log(i);  
}
```

Output

1
2
3
4
5

How it works?

- First initialize (let i = 1)
- Check condition (i <= 5)
- Run code
- Update (i++)
- Repeat until condition is false.

2. while Loop

Used when we don't know the exact number of times.

```
while (condition) {  
  // code to be executed  
}
```

Example

```
let i = 1;  
while (i <= 5) {  
  console.log(i);  
  i++;  
}
```

Output

1
2
3
4
5

Note

- Don't forget to update the value inside loop.
- Otherwise it will run forever (Infinite Loop).

3. do...while Loop

Similar to while loop, but it runs the code at least once.

```
do {  
  // code to be executed  
} while (condition);
```

Example

```
let i = 1;  
do {  
  console.log(i);  
  i++;  
} while (i <= 5);
```

Output

1
2
3
4
5

When to use?

- Use for loop when you know the count.
- Use while loop when condition is uncertain.
- Use do...while when code must run at least once.

4. break & continue

★ break

Used to stop the loop immediately.

```
for (let i = 1; i <= 5; i++) {  
  if (i === 3) break;  
  console.log(i);  
}
```

Output

1
2

Loop stops when i is 3.

★ continue

Used to skip the current iteration.

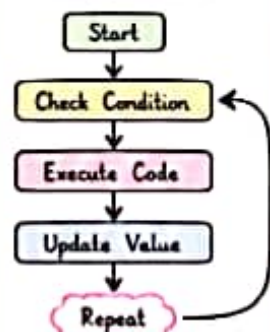
```
for (let i = 1; i <= 5; i++) {  
  if (i === 3) continue;  
  console.log(i);  
}
```

Output

1
2
4
5

3 is skipped, loop continues.

Loop Flow



Quick Notes

- Infinite loops can crash your browser.
- Always make sure your condition will eventually be false.
- Use meaningful variable names (i, j, count, etc.).
- Loops are the heart of programming logic.

JAVASCRIPT FUNCTIONS

A function is a block of code designed to perform a particular task. It helps in code reusability and makes the program modular.

KEY POINTS

- Functions reduce repetition.
- Functions improve readability.
- Functions can take inputs (parameters) and return values.
- JS functions are first-class citizens.

1. Function Declaration

The most common way to define a function.

```
function functionName(parameters) {  
  // code to be executed  
  return value; // optional  
}
```

Example

```
function add(a, b) {  
  return a + b;  
}  
  
let sum = add(5, 3);  
console.log(sum); // 8
```

Output

8

When to Use?

- Use when you want a named function.
- It can be called before its declaration.

2. Function Expression

A function stored in a variable.

```
const functionName = function(parameters) {  
  // code to be executed  
  return value;  
};
```

Example

```
const multiply = function(a, b) {  
  return a * b;  
};  
  
let result = multiply(4, 6);  
console.log(result); // 24
```

Output

24

Note

- Cannot be called before declaration.
- Ends with a semicolon ;

3. Arrow Function (ES6)

A shorter syntax for writing functions.

```
const functionName = (parameters) => {  
  // code to be executed  
  return value;  
};
```

Example

```
const square = (n) => {  
  return n * n;  
};  
  
let ans = square(5);  
console.log(ans); // 25
```

Output

25

Arrow Function Notes

- If only one parameter, parentheses are optional.
- If only one statement, curly braces and return are optional.

4. Function Without Parameters

```
function greet() {  
  console.log("Hello World!");  
}  
  
greet();
```

Output

Hello World!

5. Function with Default Parameters (ES6)

```
function greet(name = "Guest") {  
  console.log(`Hello, ${name}!`);  
}  
  
greet(); // Hello, Guest!  
greet("Shainaz"); // Hello, Shainaz!
```

Output

Hello, Guest!
Hello, Shainaz!

6. Return Statement

The return statement ends the function and returns a value.

```
function getCube(n) {  
  return n * n * n;  
}  
  
let cube = getCube(3);  
console.log(cube); // 27
```

Output

27

Quick Notes

- Functions make code reusable.
- Use meaningful names for functions.
- Keep functions small and focused.
- Arrow functions do not have their own this.

Pro Tip

- Break big tasks into small functions.
- Test your function with different inputs.



JAVASCRIPT STRINGS

Strings are used to represent text in JavaScript.
They are written inside single, double or backticks.



KEY POINTS

- Strings are immutable (cannot be changed).
- JavaScript strings are zero or more characters.
- Use useful methods to manipulate strings.



1. Creating Strings

- 'Single quotes'
- "Double quotes"
- Backticks (Template Literals)

Example

```
let a = 'Hello';
let b = "JavaScript";
let c = `I love JS!`; // template literal
```

Note

- Backticks allow multi-line strings and expression interpolation.



2. String Properties

The **length** property returns the length of a string.

Example

```
let text = "Hello World";
console.log(text.length); // 11
```

Output

11

Did You Know?

Strings in JS are indexed. First character index is 0.

Example :

```
text[0] = 'H'
text[6] = 'W'
```



3. Common String Methods

Method	Description	Example	Output
toUpperCase()	Converts string to uppercase.	"hello".toUpperCase()	HELLO
toLowerCase()	Converts string to lowercase	"HELLO".toLowerCase()	hello
trim()	Removes whitespace from both ends.	" hi ".trim()	"hi"
slice(start, end)	Extracts a part of string	"JavaScript".slice(0, 4)	"Java"
substring(start, end)	Extracts characters between indices	"JavaScript".substring(4, 10)	"Script"
replace(old, new)	Replaces a value in string	"I love JS".replace("JS", "JavaScript")	"I love JavaScript"
includes(value)	Checks if a string contains value	"Hello World".includes("World")	true
startsWith(value)	Checks if string starts with value	"Hello JS".startsWith("Hello")	true
endsWith(value)	Checks if string ends with value	"Hello JS".endsWith("JS")	true

Example

```
let msg = " Hello JavaScript ";
console.log(msg.trim()); // Hello JavaScript
console.log(msg.toUpperCase()); // HELLO JAVASCRIPT
console.log(msg.includes("JS")); // true
console.log(msg.slice(6, 10)); // Java
```

Output

```
Hello JavaScript
HELLO JAVASCRIPT
true
Java
```

Quick Notes

- Strings are immutable.
- Use backticks for multi-line strings.
- Use interpolation with \$(expression) in backticks.
- Use string methods to manipulate text easily.



Example - Template Literals

```
let name = "Shainaz";
let age = 21;
let info = `My name is ${name}
and I am ${age} years old.`;
console.log(info);
```



Template Literal Output

My name is Shainaz
and I am 21 years old.



JAVASCRIPT DOM & EVENTS

DOM (Document Object Model) represents the structure of a web page so that JavaScript can change content, style, and structure dynamically.

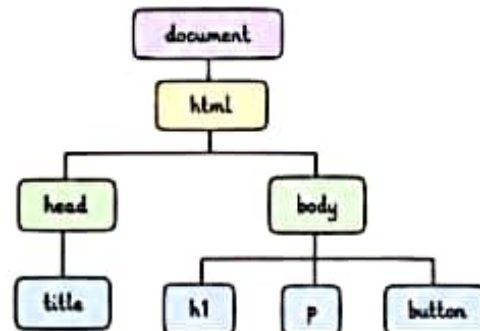
KEY POINTS

- DOM connects HTML and JS.
- JS can access and modify HTML elements.
- Events allow us to run code when something happens.

1. What is DOM?

DOM is a tree-like structure of HTML elements. Each element is an object that JS can access and modify.

Example (HTML to DOM Tree)



Common DOM Selectors

- `document.getElementById(id)`
// Selects element by ID
- `document.getElementsByClassName(class)`
// Selects elements by class
- `document.getElementsByTagName(tag)`
// Selects elements by tag
- `document.querySelector(selector)`
// Selects first element
- `document.querySelectorAll(selector)`
// Selects all matching elements

2. Accessing DOM Elements

Method	Description	Example
<code>getElementById()</code>	Selects single element by ID	<code>document.getElementById("title")</code> // <code><h1 id="title"></code>
<code>getElementsByClassName()</code>	Selects all elements by class	<code>document.getElementsByClassName("box")</code> // <code>HTMLCollection</code>
<code>getElementsByTagName()</code>	Selects all elements by tag	<code>document.getElementsByTagName("p")</code> // <code>HTMLCollection</code>
<code>querySelector()</code>	Selects first element using CSS selector	<code>document.querySelector(".box")</code> // First <code>.box</code> element
<code>querySelectorAll()</code>	Selects all elements using CSS selector	<code>document.querySelectorAll(".box")</code> // <code>NodeList</code> of all <code>.box</code>

Example

```
<!-- HTML -->
<h1 id="title" class="box">Hello JS</h1>
<p class="box">Paragraph 1</p>
<p class="box">Paragraph 2</p>

// JavaScript
let title = document.getElementById("title");
let boxes = document.getElementsByClassName("box");
let firstBox = document.querySelector(".box");
let allBoxes = document.querySelectorAll(".box");
```

3. Changing DOM Elements

Property / Method	Description	Example
<code>innerHTML</code>	Gets or sets HTML content	<code>element.innerHTML = "Hi";</code>
<code>textContent</code>	Gets or sets text (content only)	<code>element.textContent = "Hello";</code>
<code>style.property</code>	Changes CSS style	<code>element.style.color = "red";</code>
<code>setAttribute()</code>	Sets an attribute	<code>element.setAttribute("class", "box");</code>
<code>classList.add()</code>	Adds a class	<code>element.classList.add("active");</code>
<code>classList.remove()</code>	Removes a class	<code>element.classList.remove("active");</code>
<code>classList.toggle()</code>	Toggles a class	<code>element.classList.toggle("active");</code>

Example

```
let title = document.getElementById("title");
title.innerHTML = "<i>Welcome</i>";
title.textContent = "Welcome to JavaScript";
title.style.color = "blue";
title.setAttribute("data-info", "heading");

title.classList.add("active");
title.classList.remove("old");
title.classList.toggle("active");
```

Quick Notes

- DOM allows JS to interact with HTML.
- We can select, change, add or remove elements.
- Events help us run code when users interact. 😊

Mini Tip

Always use `querySelector()` and `querySelectorAll()` for modern, fast and flexible code.

JAVASCRIPT EVENTS

Events are actions that happen in the browser. JavaScript can listen for these events and run code when they occur.

KEY POINTS

- Events are actions or occurrences that happen in the browser.
- JS can listen for events and react to them.
- We use events to make web pages interactive.

1. What is an Event?

An event is something that happens to an HTML element.

Example: A button click, mouse move, key press, page load, etc.

Common Examples



How it Works?

- 1 Event occurs (User does something)
- 2 JavaScript listens for the event
- 3 JS runs the code (Handles the event)
- 4 Something happens on the page

2. Adding Events

We can add events using 3 ways:

- Inline HTML Events
- Internal JS Events (Use onclick, onchange, etc.)
- addEventListener() Method (Best Practice)

Inline HTML Events

```
<button onclick="alert('Hi!')">
  Click Me
</button>
```

addEventListener() (Best)

```
<button id="btn">Click Me</button>
<script>
  document.getElementById("btn")
    .addEventListener("click", function(){
      alert("Hello!");
    });
</script>
```

Internal JS Events

```
<button id="btn">Click Me</button>
<script>
  document.getElementById("btn").onclick
    = function(){
      alert("Hello!");
    };
</script>
```

3. Common Events

Event	Description	Example
click	Occurs when element is clicked	button.addEventListener("click", myFunc);
mouseover	Occurs when mouse pointer moves over an element	element.addEventListener("mouseover", myFunc);
mouseout	Occurs when mouse pointer moves out of an element	element.addEventListener("mouseout", myFunc);
keypress	Occurs when a key is pressed	document.addEventListener("keypress", myFunc);
submit	Occurs when a form is submitted	form.addEventListener("submit", myFunc);
change	Occurs when value of an element changes	input.addEventListener("change", myFunc);
load	Occurs when page or resource is loaded	window.addEventListener("load", myFunc);

Example (Click Event)

```
<button id="btn">Click Me</button>
<p id="msg"></p>
<script>
  document.getElementById("btn")
    .addEventListener("click", function(){
      document.getElementById("msg")
        .textContent = "Button Clicked!";
    });
</script>
```

Quick Notes

- Events make our webpage interactive.
- Use addEventListener() for clean code.
- Always test your events in the browser.

Pro Tip

Use meaningful function names when handling events. It makes your code easy to read and maintain.

JAVASCRIPT EVENTS

Events help us run code when something happens.
They make our web pages interactive.

KEY POINTS

- Events make web pages interactive.
- JS listens for events and responds to them.
- We can add events to any HTML element.

4. Event Object

When an event occurs, an event object is created automatically.
It contains helpful information about the event.

Example: `event.target`,
`event.type`

Common Properties

Property	Description
<code>type</code>	Returns the type of event
<code>target</code>	Returns the element that triggered the event
<code>currentTarget</code>	Returns the element that the listener is attached to
<code>preventDefault()</code>	Prevents the default action of the event
<code>stopPropagation()</code>	Stops the event from bubbling up the DOM

Example

```
<button id="btn">Click Me</button>
<script>
  document.getElementById("btn")
    .addEventListener("click", function(event){
      console.log("Event Type :"; event.type);
      console.log("Clicked Element "; event.target);
    });
</script>

// Output in Console
Event Type : click
Clicked Element : button#btn
```

5. Event Bubbling

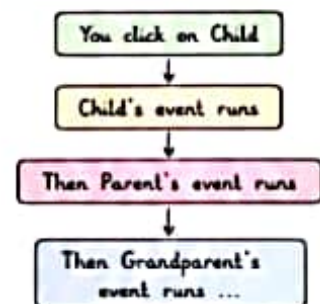
Events bubble up from the target element to its parent and ancestors.

We can stop bubbling using `stopPropagation()`.

Bubbling Example

```
<div id="parent" style="padding:20px;
border:2px solid purple;">
  Parent
  <button id="child">Click Me</button>
</div>
<script>
  document.getElementById("parent")
    .addEventListener("click", function(){
      alert("Parent Clicked");
    });
  document.getElementById("child")
    .addEventListener("click", function(event){
      alert("Child Clicked");
      // event.stopPropagation(); // try this
    });
</script>
```

How Bubbling Works?



Use `event.stopPropagation()` to stop the event from bubbling to parent elements.

6. Event Delegation

Instead of adding event to many child elements, we add the event to their parent.

It works because of event bubbling.
It is more efficient!

Example (List Items)

```
<ul id="list">
  <li>Apple</li>
  <li>Banana</li>
  <li>Mango</li>
</ul>
<script>
  document.getElementById("list")
    .addEventListener("click", function(event){
      if(event.target.tagName === "LI"){
        alert("You clicked: " + event.target.textContent);
      }
    });
</script>
```

Output

```
You clicked: Apple
You clicked: Banana
You clicked: Mango
```

Why use Delegation?

- Uses less memory
- Faster performance
- Works for dynamic elements too!

Quick Notes

- ★ Events make websites dynamic and interactive.
- ★ Use `addEventListener()` for modern code.
- ★ Understand bubbling and delegation.
- ★ Practice events to become better in JS!

Remember

- 🕒 Events = User actions
- 👂 `addEventListener()` = Listen for events
- 📄 event object = Information about the event
- 📡 Bubbling = Event moves upwards in DOM

JAVASCRIPT EVENTS

Events make our web pages interactive. They are actions or occurrences that happen in the browser.

KEY POINTS

- Events are actions in the browser.
- JS listens for events and responds to them.
- They help us create dynamic and interactive web pages.

7. Event Listener Options

We can pass an options object while adding an event listener.

Common options:

- **once**: The event runs only once.
- **capture**: The event runs in capture phase.
- **passive**: Tells the browser that the event will not call `preventDefault()`.

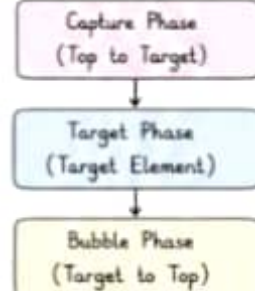
Example (once Option)

```
<button id="btn">Click Me Once</button>
<script>
  const btn = document.getElementById("btn");
  btn.addEventListener("click", function(){
    alert("Hello! I will run only once.");
  }, { once: true });
</script>
```

Output

The alert will appear only the first time you click the button.

Event Flow (Capture & Bubble)



Events first move from the top (window) down to the target, then bubble back up.

8. Removing Event Listeners

We can remove an event listener using `removeEventListener()`.

Example

```
<button id="btn">Add</button>
<button id="remove">Remove</button>
<script>
  const btn = document.getElementById("btn");
  const removeBtn = document.getElementById("remove");

  function sayHi(){
    alert("Hi! I am an event.");
  }

  btn.addEventListener("click", sayHi);
  removeBtn.addEventListener("click", function(){
    btn.removeEventListener("click", sayHi);
    alert("Event Listener Removed!");
  });
</script>
```

Output

Click "Add" → Alert shows.
Click "Remove" → Listener is removed.
Now clicking "Add" will not show alert.

Note

The same function reference must be used to remove the event listener.

9. Real World Example

Let's create a simple interactive button that changes text when clicked.

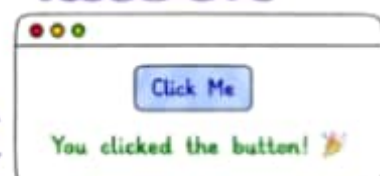
CLICK ME

Example

```
<button id="change">Click Me</button>
<p id="text">Original Text</p>
<script>
  const btn = document.getElementById("change");
  const p = document.getElementById("text");

  btn.addEventListener("click", function(){
    p.textContent = "You clicked the button! 🎉";
    p.style.color = "green";
    p.style.fontWeight = "bold";
  });
</script>
```

Output (After Click)



Quick Notes

- Events make websites interactive.
- Use `addEventListener()` to listen.
- Understand event flow.
- Remove listeners to optimize code.